```
from google.colab import drive
import pandas as pd

drive.mount('/content/drive')

file_path = '/content/cancer dataset.csv'   # Corrected path to the dataset
df = pd.read_csv(file_path)
df.head()

Drive already mounted at /content/drive; to attempt to forcibly remount, call
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	s
0	842302	M	17.99	10.38	122.80	1001.0	0
1	842517	M	20.57	17.77	132.90	1326.0	0
2	84300903	M	19.69	21.25	130.00	1203.0	0
3	84348301	M	11.42	20.38	77.58	386.1	0
4	84358402	M	20.29	14.34	135.10	1297.0	0

```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-e44552d1-0a
        title="Convert this dataframe to an interactive table."
        style="display:none;">
```

```
<div id="df-d68250e7-c125-428e-b70b-6f431c97ea80">
  <button class="colab-df-quickchart" onclick="quickchart('df-d68250e7-c125-42
            title="Suggest charts"
            style="display:none;">

  </button>

  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
    }
    (() => {
      let quickchartButtonEl =
        document.querySelector('#df-d68250e7-c125-428e-b70b-6f431c97ea80 butt
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

</div>

# Step 2: Explore & clean the dataset
import pandas as pd

# 1. View the first few rows
print("🔹 Head of the dataset:")
display(df.head())

# 2. Check shape (rows, columns)
print("\n🔹 Shape of dataset:", df.shape)

# 3. Check full info (data types, null values)
print("\n🔹 Dataset info:")
df.info()
```

```
# 4. Check summary statistics
print("\n🔹 Summary statistics:")
display(df.describe(include='all'))

# 5. Check missing values
print("\n🔹 Missing values per column:")
print(df.isnull().sum())

# 6. Standardize column names (lowercase + remove spaces)
df.columns = df.columns.str.lower().str.strip().str.replace(' ', '_')
print("\n🔹 Cleaned column names:")
print(df.columns)
```

🔹 Head of the dataset:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	s
0	842302	M	17.99	10.38	122.80	1001.0	0
1	842517	M	20.57	17.77	132.90	1326.0	0
2	84300903	M	19.69	21.25	130.00	1203.0	0
3	84348301	M	11.42	20.38	77.58	386.1	0
4	84358402	M	20.29	14.34	135.10	1297.0	0

```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-e7d24967-74
        title="Convert this dataframe to an interactive table."
        style="display:none;">
```

```
<div id="df-3876cd38-486f-4042-bf01-395bf89ac6da">
  <button class="colab-df-quickchart" onclick="quickchart('df-3876cd38-486f-40
            title="Suggest charts"
            style="display:none;">

  </button>

  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
    }
    (() => {
      let quickchartButtonEl =
        document.querySelector('#df-3876cd38-486f-4042-bf01-395bf89ac6da butt
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

</div>

🔹 Shape of dataset: (569, 32)

🔹 Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
```

```
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

- Summary statistics:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	a
count	5.690000e+02	569	569.000000	569.000000	569.000000	56
unique	NaN	2	NaN	NaN	NaN	Na
top	NaN	B	NaN	NaN	NaN	Na
freq	NaN	357	NaN	NaN	NaN	Na
mean	3.037183e+07	NaN	14.127292	19.289649	91.969033	65

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	a
std	1.250206e+08	NaN	3.524049	4.301036	24.298981	35
min	8.670000e+03	NaN	6.981000	9.710000	43.790000	14
25%	8.692180e+05	NaN	11.700000	16.170000	75.170000	42
50%	9.060240e+05	NaN	13.370000	18.840000	86.240000	55
75%	8.813129e+06	NaN	15.780000	21.800000	104.100000	78
max	9.113205e+08	NaN	28.110000	39.280000	188.500000	25

```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-c50eefe2-2
        title="Convert this dataframe to an interactive table."
        style="display:none;">
```

```
<div id="df-e2ca9905-47c6-4437-a131-124238d6a356">
  <button class="colab-df-quickchart" onclick="quickchart('df-e2ca9905-47c6-4
            title="Suggest charts"
            style="display:none;">
```

```
  </button>
```

```
  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
    }
    (() => {
      let quickchartButtonEl =
        document.querySelector('#df-e2ca9905-47c6-4437-a131-124238d6a356 butto
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

</div>
```

🔹 Missing values per column:

```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
```

```
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64

 🔹 Cleaned column names:
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave_points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave_points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave_points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')

# Basic EDA - STEP 1: Show first few rows
print(" 🔹 First 5 rows of the dataset:")
display(df.head())

 🔹 First 5 rows of the dataset:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	s
0	842302	M	17.99	10.38	122.80	1001.0	0
1	842517	M	20.57	17.77	132.90	1326.0	0
2	84300903	M	19.69	21.25	130.00	1203.0	0
3	84348301	M	11.42	20.38	77.58	386.1	0
4	84358402	M	20.29	14.34	135.10	1297.0	0

```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-098c8b45-8
        title="Convert this dataframe to an interactive table."
        style="display:none;">
```

```
<div id="df-dfa83741-54bb-4c0e-b635-f57b3af1e58b">
  <button class="colab-df-quickchart" onclick="quickchart('df-dfa83741-54bb-4
            title="Suggest charts"
            style="display:none;">
```

```
  </button>

  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
    }
    (() => {
      let quickchartButtonEl =
```

```
        document.querySelector('#df-dfa83741-54bb-4c0e-b635-f57b3af1e58b butto
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

</div>

print("🔹 Dataset Shape (rows, columns):", df.shape)
```

```
🔹 Dataset Shape (rows, columns): (569, 32)
```

```
print("🔹 Dataset Info:")
df.info()
```

```
🔹 Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave_points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave_points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
```

```
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave_points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB

print("🔹 Summary Statistics:")
display(df.describe())
```

🔹 Summary Statistics:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	s
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	5
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0

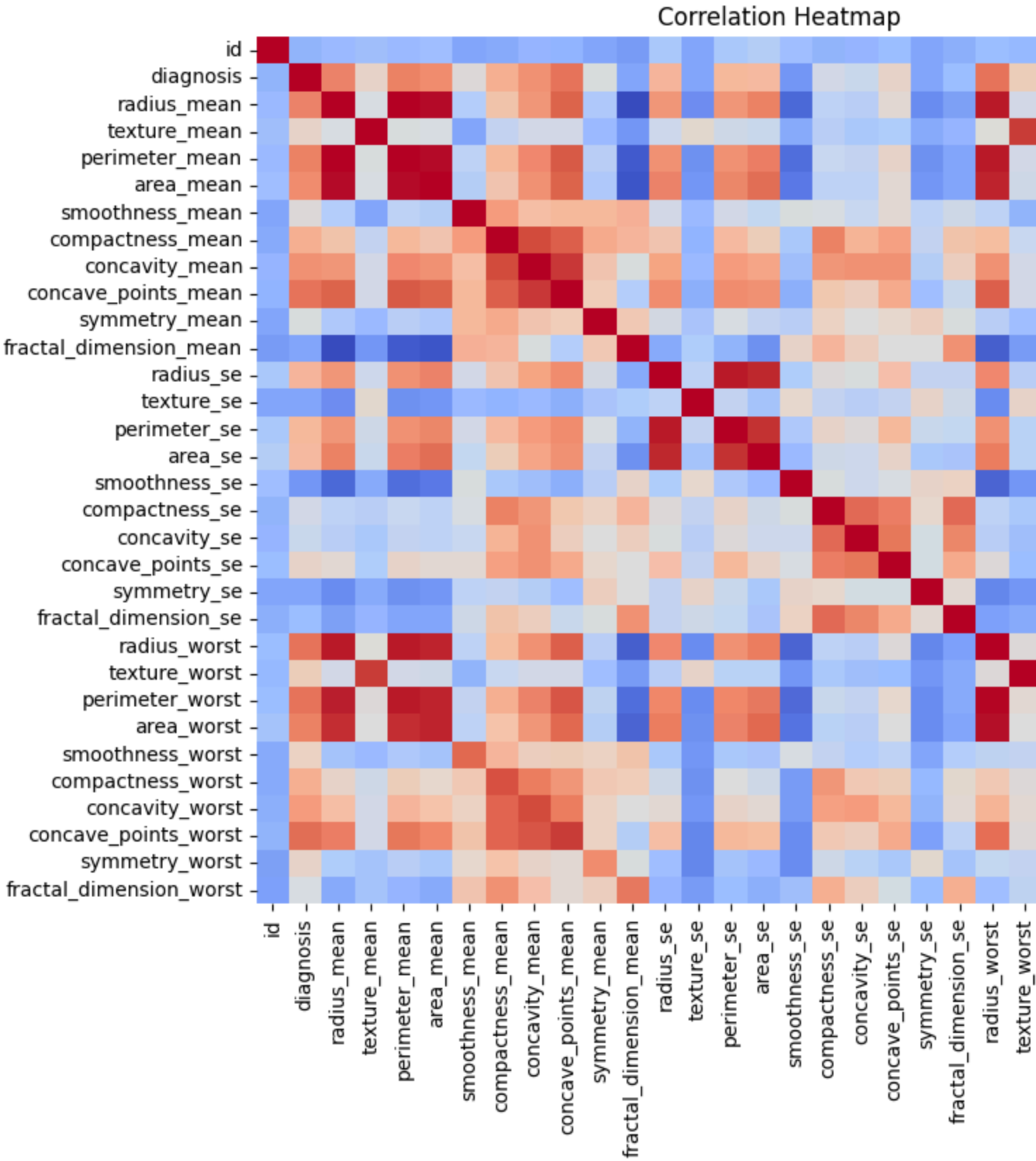
```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-a4e362fa-c
        title="Convert this dataframe to an interactive table."
        style="display:none;">
```

```
<div id="df-45ba923f-6402-44c0-ad66-175a38eb865c">
  <button class="colab-df-quickchart" onclick="quickchart('df-45ba923f-6402-44
            title="Suggest charts"
            style="display:none;">

```

```

  </button>

  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
    }
    (() => {
      let quickchartButtonEl =
        document.querySelector('#df-45ba923f-6402-44c0-ad66-175a38eb865c butt
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

</div>

print("🔹 Missing Values in Each Column:")
print(df.isnull().sum())
```

```
🔹 Missing Values in Each Column:
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave_points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave_points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave_points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64

print("🔹 Diagnosis Value Counts (Class Distribution):")
print(df['diagnosis'].value_counts())

🔹 Diagnosis Value Counts (Class Distribution):
diagnosis
B    357
M    212
Name: count, dtype: int64

import matplotlib.pyplot as plt
import seaborn as sns

# Create a copy of the DataFrame to avoid modifying the original 'df' for this
df_encoded = df.copy()
```

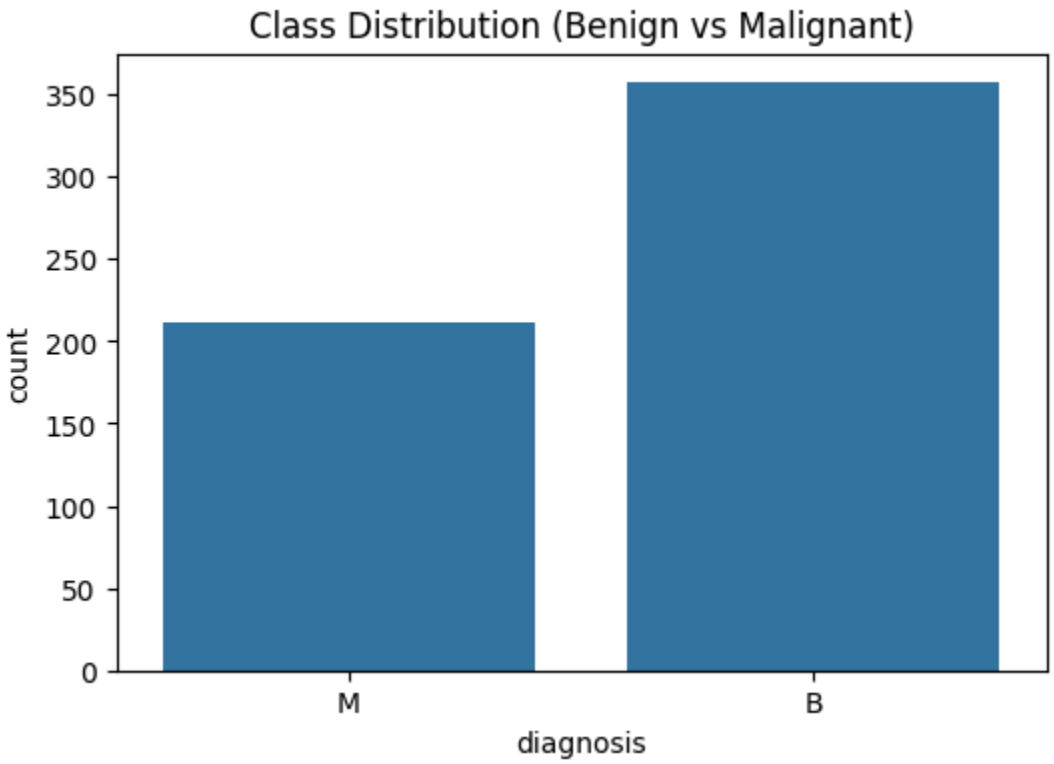
```
# Encode the 'diagnosis' column to numerical values (M=1, B=0)
df_encoded['diagnosis'] = df_encoded['diagnosis'].map({'M': 1, 'B': 0})

plt.figure(figsize=(12,8))
sns.heatmap(df_encoded.corr(), cmap='coolwarm', annot=False)
plt.title("Correlation Heatmap")
plt.show()
```



png

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
sns.countplot(data=df, x='diagnosis')
plt.title("Class Distribution (Benign vs Malignant)")
plt.show()
```



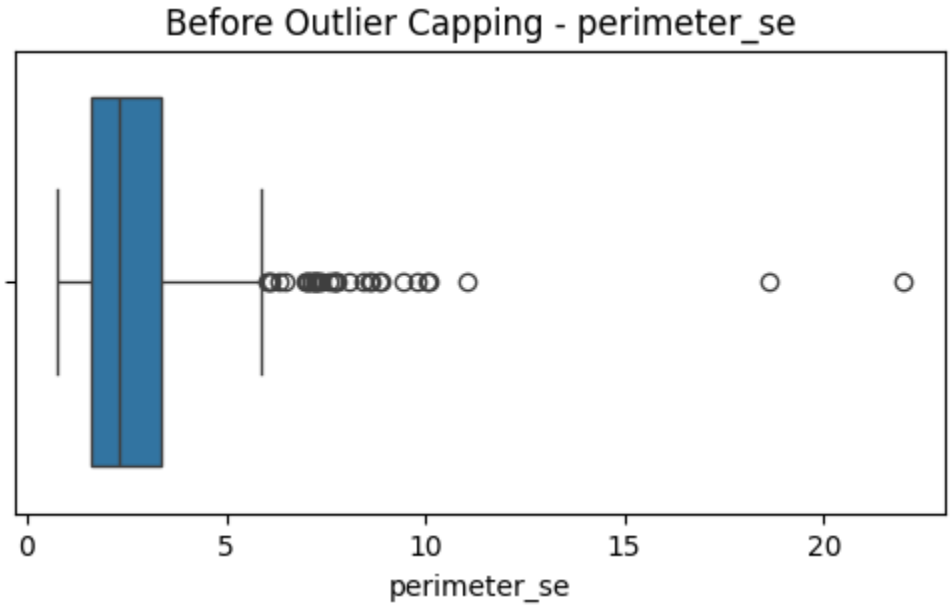
png

```
corr = df_encoded.corr()['diagnosis'].abs().sort_values(ascending=False)
print("🔹 Top 10 features correlated with diagnosis:")
print(corr.head(11))
```

```
🔹 Top 10 features correlated with diagnosis:
diagnosis               1.000000
concave_points_worst    0.793566
perimeter_worst         0.782914
concave_points_mean     0.776614
radius_worst            0.776454
perimeter_mean          0.742636
area_worst              0.733825
radius_mean             0.730029
area_mean               0.708984
concavity_mean          0.696360
concavity_worst         0.659610
Name: diagnosis, dtype: float64
```

```
#Quick checks (confirm variables & shapes)
#Plot – Before Outlier Capping (boxplot for one column)
#Re-apply capping on training split (if not already) – use same caps on test
#Plot – After Outlier Capping (boxplot same column)
#Plot – Feature distribution Before Scaling (hist for one feature)
#Scale features (StandardScaler)
#Plot – Feature distribution After Scaling (hist for same feature)
#Apply SMOTE on training set (if not already)
#Plot – Class balance after SMOTE (countplot)
#Optional: Save scaler and show final shapes


# Step 0: Train-test split (run if you haven't yet)
from sklearn.model_selection import train_test_split

# Ensure diagnosis encoded as 0/1
assert 'diagnosis' in df.columns, "diagnosis column missing"
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("Train/test created:", X_train.shape, X_test.shape, y_train.shape, y_te


Train/test created: (455, 31) (114, 31) (455,) (114,)

# Step 1: Quick checks
import numpy as np

print("df shape:", df.shape)
print("Columns present:", df.columns.tolist()[:10], "...")
for col in ['perimeter_se','radius_mean','compactness_se']:
    print(col, "in df?", col in df.columns)

df shape: (569, 32)
Columns present: ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimete
perimeter_se in df? True
radius_mean in df? True
compactness_se in df? True

# Step 2: Boxplot BEFORE capping for a selected column
import seaborn as sns
import matplotlib.pyplot as plt

col = 'perimeter_se'   # change to any column you want to visualize
if col in X_train.columns:
```

```
    plt.figure(figsize=(6,3))
    sns.boxplot(x=X_train[col])
    plt.title(f"Before Outlier Capping - {col}")
    plt.show()
else:
    print(f"Column {col} not found in X_train.")
```

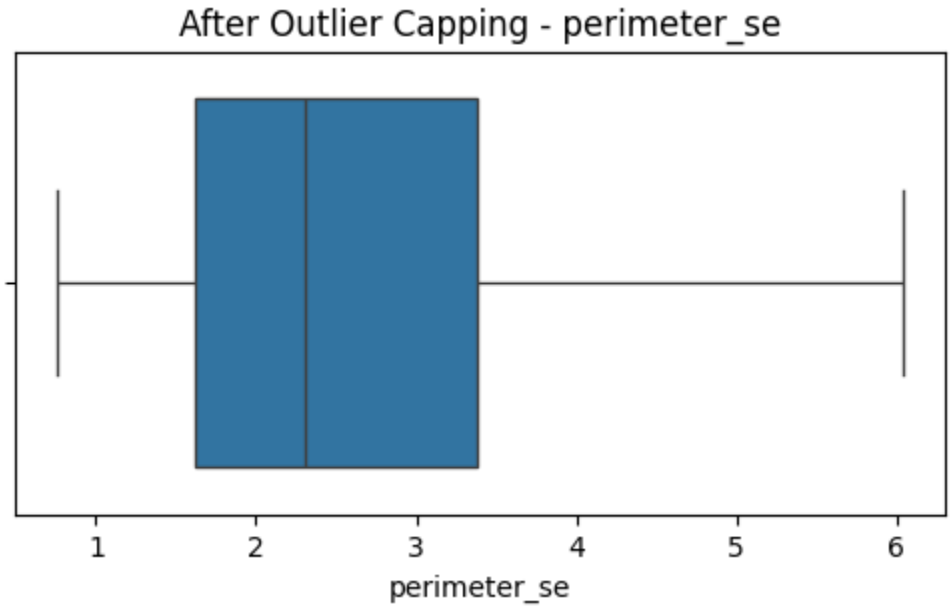


png

```
# Step 3: Apply capping (IQR) for selected outlier columns on train & test
import numpy as np

outlier_cols = ['perimeter_se','compactness_se','concavity_se','radius_worst'
for col in outlier_cols:
    if col in X_train.columns:
        Q1 = X_train[col].quantile(0.25)
        Q3 = X_train[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        X_train[col] = np.clip(X_train[col], lower, upper)
        X_test[col]  = np.clip(X_test[col], lower, upper)
        print(f"Capped {col}: lower={lower:.4f}, upper={upper:.4f}")
    else:
        print(f"Skip {col}: not in X_train")
print("Capping applied.")
```

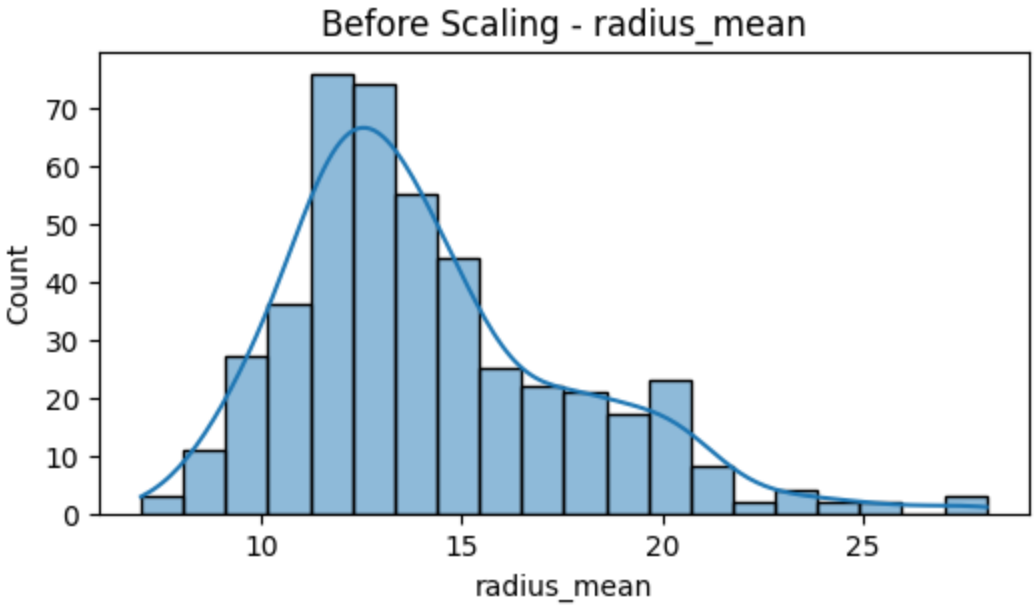
```
Capped perimeter_se: lower=-1.0428, upper=6.0393
Capped compactness_se: lower=-0.0155, upper=0.0606
Capped concavity_se: lower=-0.0232, upper=0.0787
Capped radius_worst: lower=3.7725, upper=28.4325
```

```
Capped compactness_worst: lower=-0.1378, upper=0.6236
Capping applied.

# Step 4: Boxplot AFTER capping for the same column
col = 'perimeter_se'   # keep same col used previously
if col in X_train.columns:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=X_train[col])
    plt.title(f"After Outlier Capping - {col}")
    plt.show()
else:
    print(f"Column {col} not found in X_train.")
```



png

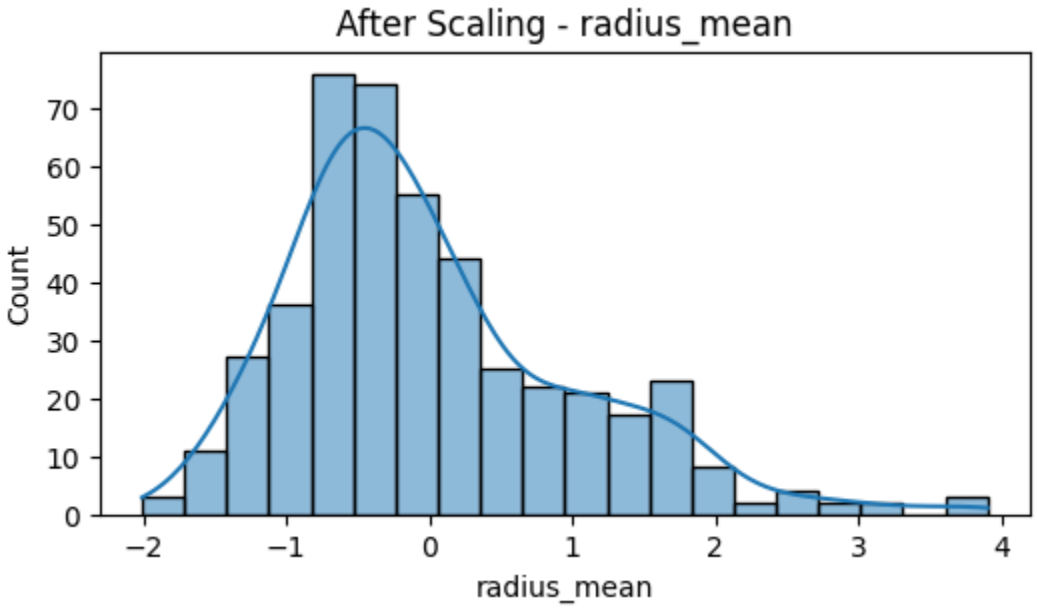
```
# Step 5: Histogram before scaling for a representative feature
feature = 'radius_mean'
if feature in X_train.columns:
    plt.figure(figsize=(6,3))
    sns.histplot(X_train[feature], kde=True)
    plt.title(f"Before Scaling - {feature}")
    plt.show()
else:
    print(f"{feature} not found in X_train.")
```



png

```
# STEP 6 — Scaling (StandardScaler) — FIXED VERSION
from sklearn.preprocessing import StandardScaler
import pandas as pd

scaler = StandardScaler()

# 1. Fit-transform on training
X_train_scaled = scaler.fit_transform(X_train)

# 2. Transform on test
X_test_scaled = scaler.transform(X_test)

# 3. Convert BACK to DataFrame (IMPORTANT)
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns, inde
X_test_scaled_df  = pd.DataFrame(X_test_scaled,  columns=X_test.columns,  inde

print("Scaling done successfully.")
print("X_train_scaled_df shape:", X_train_scaled_df.shape)
print("X_test_scaled_df shape:", X_test_scaled_df.shape)

Scaling done successfully.
X_train_scaled_df shape: (455, 31)
X_test_scaled_df shape: (114, 31)

# Step 7: Histogram after scaling for the same feature
feature = 'radius_mean'
if feature in X_train_scaled_df.columns:
    plt.figure(figsize=(6,3))
    sns.histplot(X_train_scaled_df[feature], kde=True)
```

```
    plt.title(f"After Scaling - {feature}")
    plt.show()
else:
    print(f"{feature} not in scaled dataframe.")
```

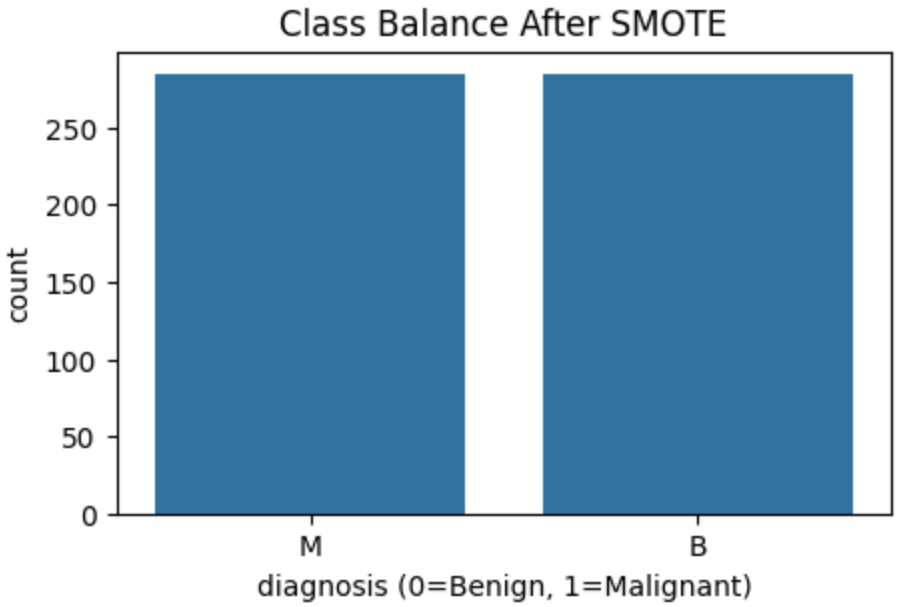


png

```
# Step 8: Apply SMOTE to training set
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train_scaled, y_train)

print("After SMOTE shapes:", X_train_sm.shape, y_train_sm.shape)
```

```
After SMOTE shapes: (570, 31) (570,)
```

```
# Step 9: Plot class balance after SMOTE
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

plt.figure(figsize=(5,3))
sns.countplot(x=y_train_sm)
plt.title("Class Balance After SMOTE")
plt.xlabel("diagnosis (0=Benign, 1=Malignant)")
plt.show()

# Also print counts
print("Counts after SMOTE:")
print(pd.Series(y_train_sm).value_counts())
```



png

```
Counts after SMOTE:
diagnosis
M    285
B    285
Name: count, dtype: int64

# STEP A — COMMON IMPORTS + METRIC FUNCTION

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

def evaluate_model(name, y_true, y_pred, y_proba):
    print(f"\n==================== {name} ====================")
    print(f"Accuracy  : {accuracy_score(y_true, y_pred):.4f}")
    print(f"Precision : {precision_score(y_true, y_pred):.4f}")
    print(f"Recall    : {recall_score(y_true, y_pred):.4f}")
    print(f"F1 Score  : {f1_score(y_true, y_pred):.4f}")
    print(f"ROC AUC   : {roc_auc_score(y_true, y_proba):.4f}")

    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
# Step 10: Save scaler for later use and print final shapes
```

```
import joblib
joblib.dump(scaler, 'scaler.pkl')
print("Scaler saved to scaler.pkl")

print("Final dataset shapes:")
print("X_train_sm:", X_train_sm.shape)
print("y_train_sm:", y_train_sm.shape)
print("X_test_scaled:", X_test_scaled.shape)
print("y_test:", y_test.shape)
```

```
Scaler saved to scaler.pkl
Final dataset shapes:
X_train_sm: (570, 31)
y_train_sm: (570,)
X_test_scaled: (114, 31)
y_test: (114,)
```

```
print(y_test.unique())
print(y_train_sm[:10])
```

```
['B' 'M']
0    M
1    B
2    B
3    M
4    M
5    M
6    B
7    M
8    M
9    M
Name: diagnosis, dtype: object
```

```
# FIX: Convert ALL diagnosis columns everywhere to numeric (0 = B, 1 = M)

# For original df
df['diagnosis'] = df['diagnosis'].map({'B':0, 'M':1})

# For y_test (pandas series)
y_test = y_test.map({'B':0, 'M':1})

# For y_train (before SMOTE)
y_train = y_train.map({'B':0, 'M':1})

# For SMOTE output y_train_sm (numpy array or list)
y_train_sm = pd.Series(y_train_sm).map({'B':0, 'M':1}).values

print("df diagnosis unique:", df['diagnosis'].unique())
print("y_test unique:", y_test.unique())
```

```
print("y_train_sm sample:", y_train_sm[:10])
```

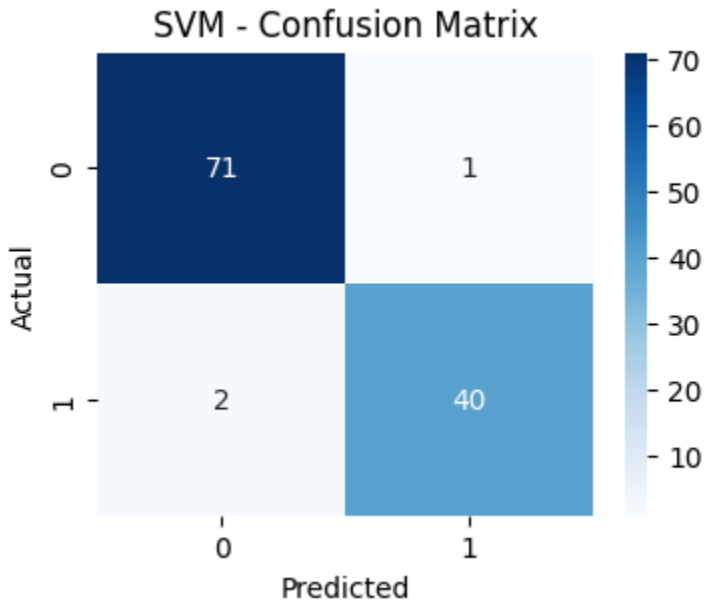
```
df diagnosis unique: [1 0]
y_test unique: [0 1]
y_train_sm sample: [1 0 0 1 1 1 0 1 1 1]
```

```
# STEP A — Imports + Evaluation + Graph Functions

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

def evaluate_model(name, y_true, y_pred, y_proba):
    print(f"\n==================== {name} ====================")
    print(f"Accuracy  : {accuracy_score(y_true, y_pred):.4f}")
    print(f"Precision : {precision_score(y_true, y_pred):.4f}")
    print(f"Recall    : {recall_score(y_true, y_pred):.4f}")
    print(f"F1 Score  : {f1_score(y_true, y_pred):.4f}")
    print(f"ROC AUC   : {roc_auc_score(y_true, y_proba):.4f}")

def plot_confusion_matrix(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(4,3))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"{model_name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

def plot_roc_curve(y_true, y_proba, model_name):
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    auc_val = auc(fpr, tpr)

    plt.figure(figsize=(4,3))
    plt.plot(fpr, tpr, label=f"AUC = {auc_val:.3f}")
    plt.plot([0,1],[0,1],'k--')
    plt.title(f"{model_name} - ROC Curve")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.legend()
    plt.show()

print("STEP A DONE ✔")
```

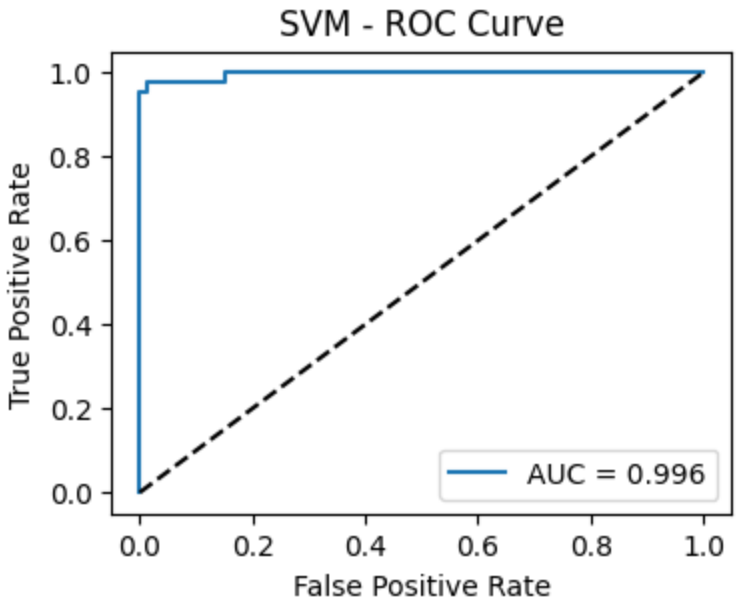
```
STEP A DONE ✔
```

```
# STEP B — SVM
```

```
from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', probability=True, random_state=42)
svm_model.fit(X_train_sm, y_train_sm)

svm_pred = svm_model.predict(X_test_scaled)
svm_proba = svm_model.predict_proba(X_test_scaled)[:, 1]

# evaluation
evaluate_model("SVM", y_test, svm_pred, svm_proba)

# GRAPH 1: Confusion Matrix
plot_confusion_matrix(y_test, svm_pred, "SVM")

# GRAPH 2: ROC Curve
plot_roc_curve(y_test, svm_proba, "SVM")

print("SVM DONE ✔")

==================== SVM ====================
Accuracy  : 0.9737
Precision : 0.9756
Recall    : 0.9524
F1 Score  : 0.9639
ROC AUC   : 0.9960
```

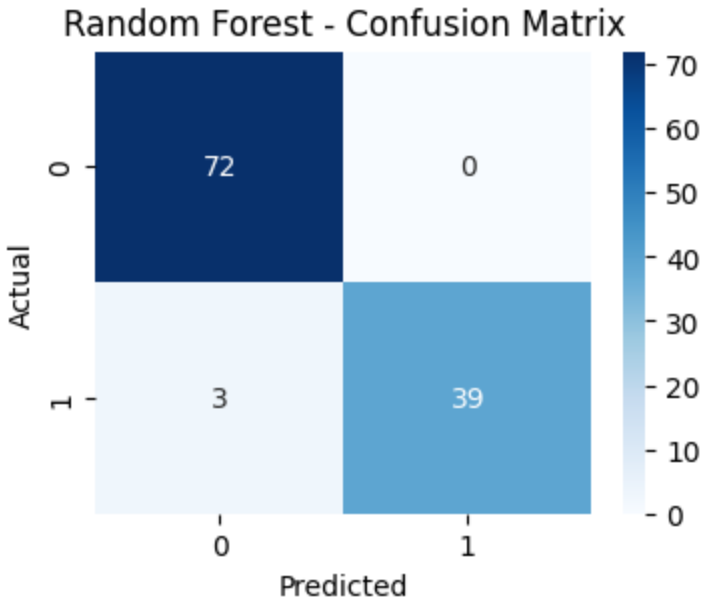


png

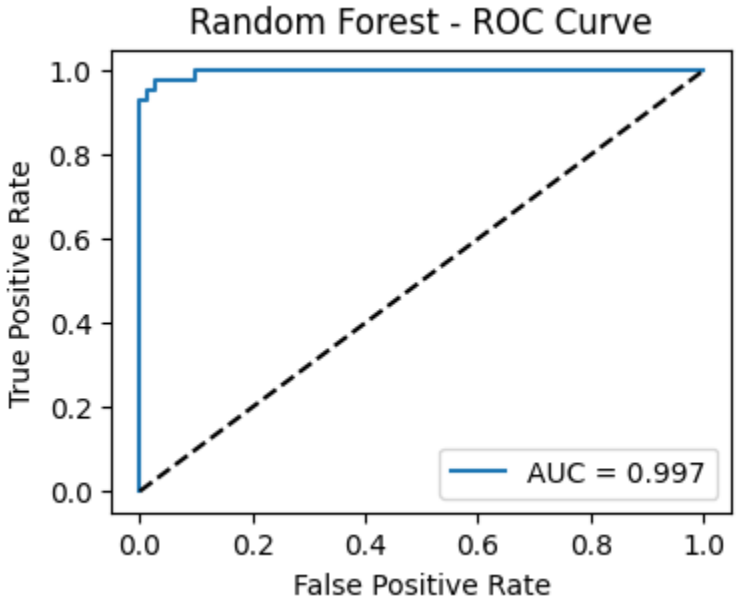


png

```
SVM DONE ✔

# STEP C — RANDOM FOREST

from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_sm, y_train_sm)

rf_pred = rf_model.predict(X_test_scaled)
rf_proba = rf_model.predict_proba(X_test_scaled)[:, 1]

# evaluation
evaluate_model("Random Forest", y_test, rf_pred, rf_proba)

# GRAPH 1
plot_confusion_matrix(y_test, rf_pred, "Random Forest")

# GRAPH 2
plot_roc_curve(y_test, rf_proba, "Random Forest")

print("RANDOM FOREST DONE ✔")

==================== Random Forest ====================
Accuracy  : 0.9737
Precision : 1.0000
Recall    : 0.9286
F1 Score  : 0.9630
ROC AUC   : 0.9967
```



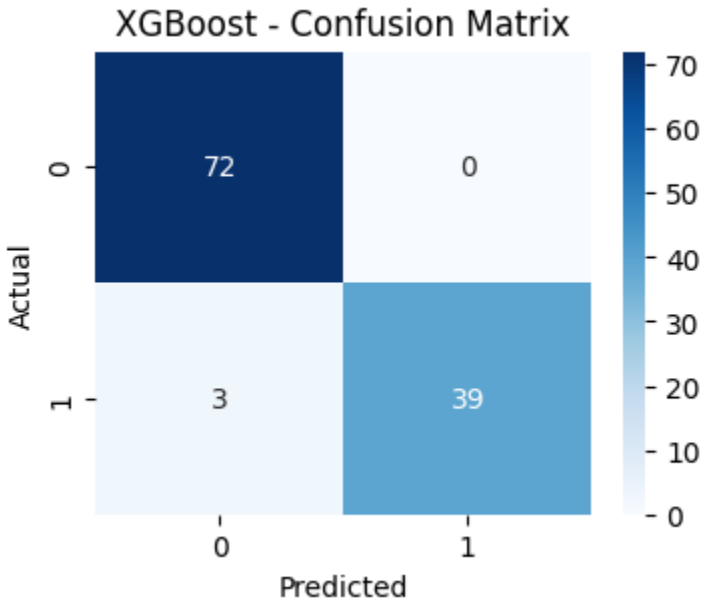
png



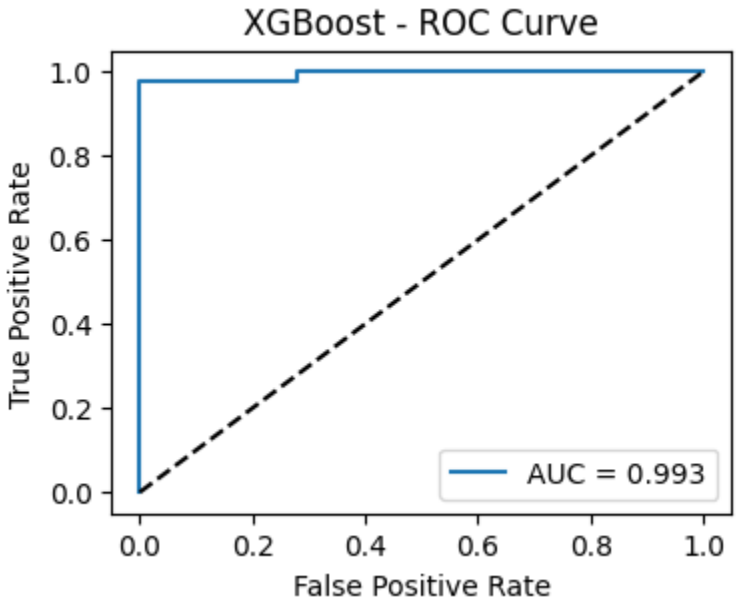
png

```
RANDOM FOREST DONE ✔

# STEP D — XGBOOST

import xgboost as xgb

xgb_model = xgb.XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
```

```
xgb_model.fit(X_train_sm, y_train_sm)

xgb_pred = xgb_model.predict(X_test_scaled)
xgb_proba = xgb_model.predict_proba(X_test_scaled)[:, 1]

# evaluation
evaluate_model("XGBoost", y_test, xgb_pred, xgb_proba)

# GRAPH 1
plot_confusion_matrix(y_test, xgb_pred, "XGBoost")

# GRAPH 2
plot_roc_curve(y_test, xgb_proba, "XGBoost")

print("XGBOOST DONE ✔")

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)



==================== XGBoost ====================
Accuracy  : 0.9737
Precision : 1.0000
Recall    : 0.9286
F1 Score  : 0.9630
ROC AUC   : 0.9934
```



png



png

```
XGBOOST DONE ✔

# STEP E — MODEL COMPARISON

comparison_df = pd.DataFrame({
    "Model": ["SVM", "Random Forest", "XGBoost"],

    "Accuracy": [
        accuracy_score(y_test, svm_pred),
        accuracy_score(y_test, rf_pred),
        accuracy_score(y_test, xgb_pred)
    ],

    "Precision": [
        precision_score(y_test, svm_pred),
        precision_score(y_test, rf_pred),
        precision_score(y_test, xgb_pred)
    ],

    "Recall": [
        recall_score(y_test, svm_pred),
        recall_score(y_test, rf_pred),
        recall_score(y_test, xgb_pred)
    ],

    "F1 Score": [
        f1_score(y_test, svm_pred),
        f1_score(y_test, rf_pred),
```

```
        f1_score(y_test, xgb_pred)
    ],

    "ROC-AUC": [
        roc_auc_score(y_test, svm_proba),
        roc_auc_score(y_test, rf_proba),
        roc_auc_score(y_test, xgb_proba)
    ]
})

# Sort by F1 Score (best metric for medical diagnosis)
comparison_df = comparison_df.sort_values(by="F1 Score", ascending=False)

print("=== MODEL COMPARISON TABLE ===")
display(comparison_df)

=== MODEL COMPARISON TABLE ===

<div>
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	SVM	0.973684	0.97561	0.952381	0.963855	0.996032
1	Random Forest	0.973684	1.00000	0.928571	0.962963	0.996693
2	XGBoost	0.973684	1.00000	0.928571	0.962963	0.993386

```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-2b8cc6a4-a9
        title="Convert this dataframe to an interactive table."
        style="display:none;">
```

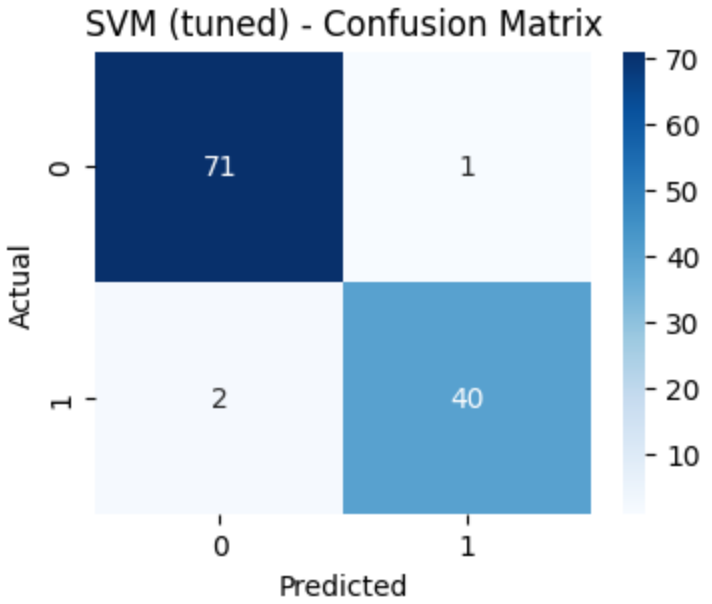
```
<div id="df-6e66c2d1-0c9e-4102-a7c7-27fda8957e7f">
  <button class="colab-df-quickchart" onclick="quickchart('df-6e66c2d1-0c9e-4
            title="Suggest charts"
            style="display:none;">
```

```
  </button>

  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
    }
    (() => {
      let quickchartButtonEl =
        document.querySelector('#df-6e66c2d1-0c9e-4102-a7c7-27fda8957e7f butt
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

<style>
  .colab-df-generate {
    background-color: #E8F0FE;
    border: none;
    border-radius: 50%;
    cursor: pointer;
    display: none;
```

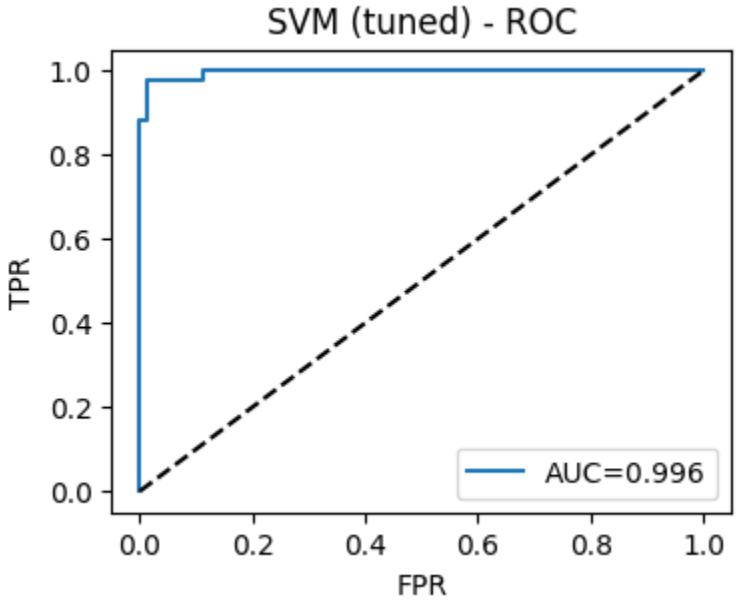
```
    fill: #1967D2;
    height: 32px;
    padding: 0 0 0 0;
    width: 32px;
  }

  .colab-df-generate:hover {
    background-color: #E2EBFA;
    box-shadow: 0px 1px 2px rgba(60, 64, 67, 0.3), 0px 1px 3px 1px rgba(60, 64
    fill: #174EA6;
  }

  [theme=dark] .colab-df-generate {
    background-color: #3B4455;
    fill: #D2E3FC;
  }

  [theme=dark] .colab-df-generate:hover {
    background-color: #434B5C;
    box-shadow: 0px 1px 3px 1px rgba(0, 0, 0, 0.15);
    filter: drop-shadow(0px 1px 2px rgba(0, 0, 0, 0.3));
    fill: #FFFFFF;
  }
</style>
<button class="colab-df-generate" onclick="generateWithVariable('comparison_df
        title="Generate code using this dataframe."
        style="display:none;">

✨

</div>

best_model = comparison_df.iloc[0]["Model"]
print("🔥 Best Performing Model:", best_model)

🔥 Best Performing Model: SVM

#Plan (short)

#Hyperparameter tuning for SVM (RandomizedSearchCV)

#Evaluate tuned SVM on test set (metrics + 2 graphs)

#Get feature importances via permutation importance (since SVM isn't directly

#Retrain SVM on top-k features (compare performance)

# SHAP explanations for selected instances
```

```
#Save final model + scaler

# STEP 1: Hyperparameter tuning for SVM (RandomizedSearchCV)
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.svm import SVC
import numpy as np
import scipy.stats as stats
import joblib

# Create SVC
svc = SVC(probability=True, random_state=42)

# Distributions
param_dist = {
    'C': stats.loguniform(1e-3, 1e3),        # wide range
    'gamma': stats.loguniform(1e-4, 1e1),    # for rbf
    'kernel': ['rbf', 'poly'],               # try rbf and poly
    'degree': [2,3,4]                        # used for poly
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rs = RandomizedSearchCV(
    estimator=svc,
    param_distributions=param_dist,
    n_iter=30,
    scoring='f1',
    n_jobs=-1,
    cv=cv,
    random_state=42,
    verbose=1,
    return_train_score=False
)

# If X_train_sm is numpy but you want DataFrame mapping:
# cols = X_train.columns if defined else [f"f{i}" for i in range(X_train_sm.sh
# X_train_sm_df = pd.DataFrame(X_train_sm, columns=cols)

rs.fit(X_train_sm, y_train_sm)

print("Best params:", rs.best_params_)
print("Best CV F1:", rs.best_score_)

# Save the search object and best estimator
joblib.dump(rs, "svm_random_search.pkl")
joblib.dump(rs.best_estimator_, "svm_best_estimator.pkl")

# make best_estimator available
```

```
svm_tuned = rs.best_estimator_

Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best params: {'C': np.float64(618.7736959637081), 'degree': 2, 'gamma': np.fl
Best CV F1: 0.9770499249173235

# STEP 2: Evaluate tuned SVM on test set
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# predictions
svm_tuned_pred = svm_tuned.predict(X_test_scaled)
svm_tuned_proba = svm_tuned.predict_proba(X_test_scaled)[:,1]

# metrics
print("Accuracy :", accuracy_score(y_test, svm_tuned_pred))
print("Precision:", precision_score(y_test, svm_tuned_pred))
print("Recall   :", recall_score(y_test, svm_tuned_pred))
print("F1       :", f1_score(y_test, svm_tuned_pred))
print("ROC AUC  :", roc_auc_score(y_test, svm_tuned_proba))

# Confusion matrix
cm = confusion_matrix(y_test, svm_tuned_pred)
plt.figure(figsize=(4,3))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("SVM (tuned) - Confusion Matrix")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.show()

# ROC
fpr, tpr, _ = roc_curve(y_test, svm_tuned_proba)
plt.figure(figsize=(4,3))
plt.plot(fpr, tpr, label=f"AUC={auc(fpr,tpr):.3f}")
plt.plot([0,1],[0,1],'k--')
plt.title("SVM (tuned) - ROC")
plt.xlabel("FPR"); plt.ylabel("TPR")
plt.legend()
plt.show()

Accuracy : 0.9736842105263158
Precision: 0.975609756097561
Recall   : 0.9523809523809523
F1       : 0.963855421686747
ROC AUC  : 0.9960317460317459
```



png



png

```
# Permutation Importance for Tuned SVM
from sklearn.inspection import permutation_importance
import pandas as pd
import matplotlib.pyplot as plt

# use DataFrame with feature names
feature_names = X_train.columns.tolist()
X_test_df = pd.DataFrame(X_test_scaled, columns=feature_names)

perm = permutation_importance(
    svm_tuned,
```

```
    X_test_df,
    y_test,
    n_repeats=20,
    random_state=42,
    n_jobs=-1
)

imp_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": perm.importances_mean
}).sort_values(by="Importance", ascending=False)

print("Top 10 Features:")
display(imp_df.head(10))

# Plot top 10
plt.figure(figsize=(8,5))
plt.barh(imp_df.head(10)["Feature"][::-1], imp_df.head(10)["Importance"][::-1
plt.title("Top 10 Important Features (Permutation Importance)")
plt.xlabel("Importance Score")
plt.show()

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732: Use
  warnings.warn(


Top 10 Features:

<div>
```

	Feature	Importance
8	concave_points_mean	0.050877
17	concavity_se	0.037719
29	symmetry_worst	0.036404
22	texture_worst	0.034211
16	compactness_se	0.028070
30	fractal_dimension_worst	0.022807
11	radius_se	0.019298
12	texture_se	0.015351
21	radius_worst	0.014474
13	perimeter_se	0.014035

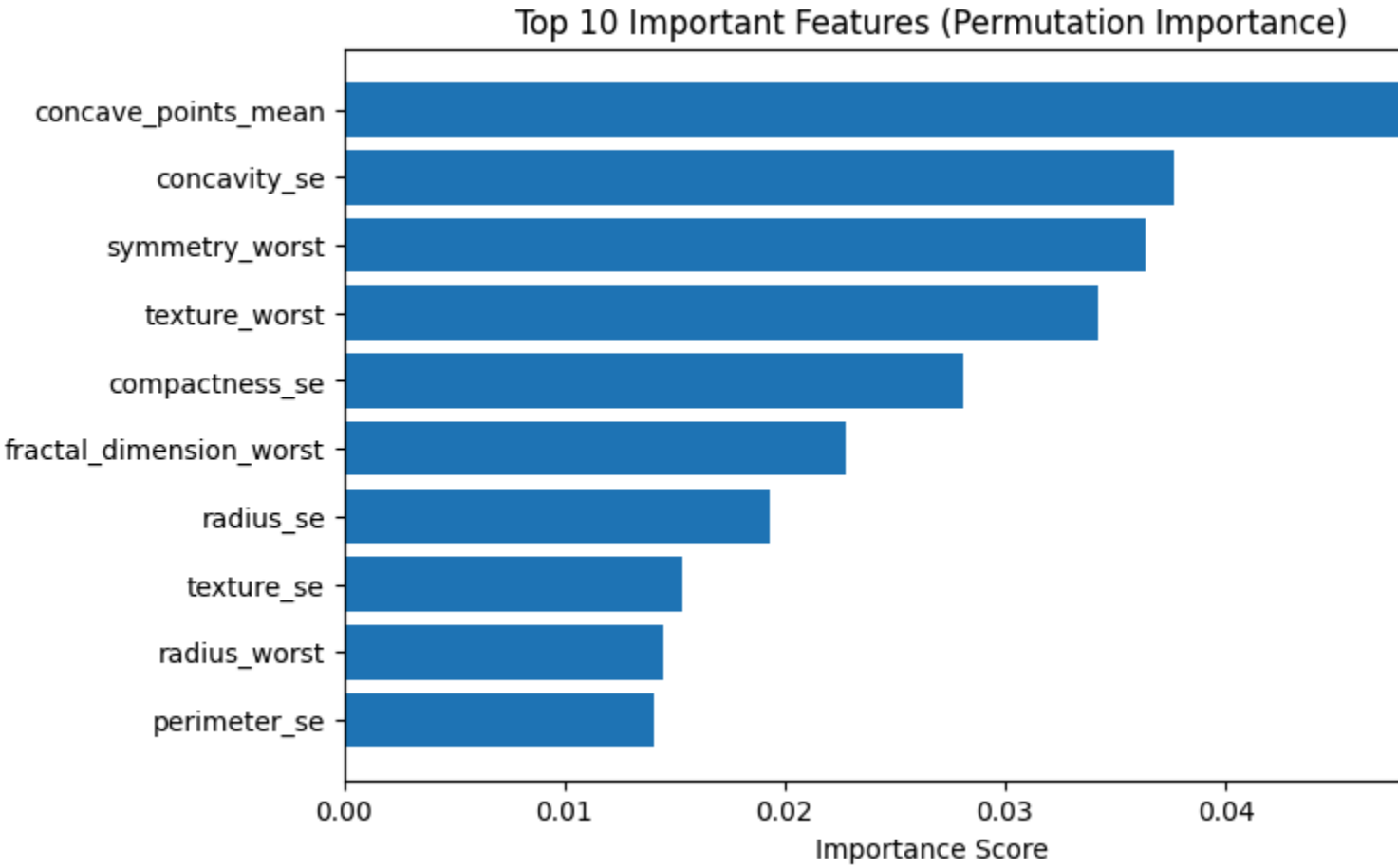
```
<div class="colab-df-buttons">


<button class="colab-df-convert" onclick="convertToInteractive('df-1ed625dc-e
        title="Convert this dataframe to an interactive table."
```

```
        style="display:none;">
```

```
<div id="df-52d7c799-8efb-4f05-94b0-18840b4dc361">
  <button class="colab-df-quickchart" onclick="quickchart('df-52d7c799-8efb-4
            title="Suggest charts"
            style="display:none;">
```

```
  </button>

  <script>
    async function quickchart(key) {
      const quickchartButtonEl =
        document.querySelector('#' + key + ' button');
      quickchartButtonEl.disabled = true;  // To prevent multiple clicks.
      quickchartButtonEl.classList.add('colab-df-spinner');
      try {
        const charts = await google.colab.kernel.invokeFunction(
            'suggestCharts', [key], {});
      } catch (error) {
        console.error('Error during call to suggestCharts:', error);
      }
      quickchartButtonEl.classList.remove('colab-df-spinner');
      quickchartButtonEl.classList.add('colab-df-quickchart-complete');
```

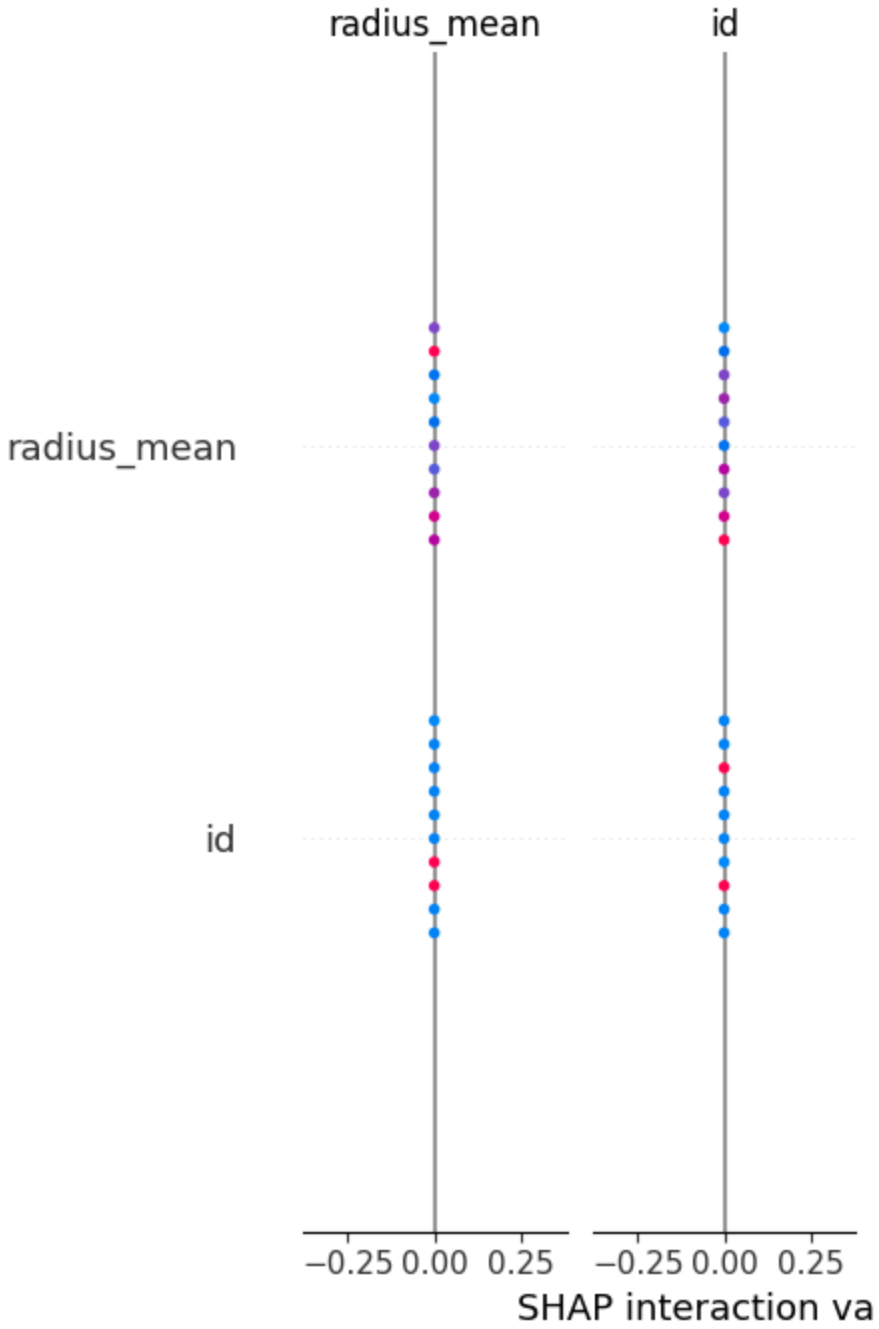
```
    }
    (() => {
      let quickchartButtonEl =
        document.querySelector('#df-52d7c799-8efb-4f05-94b0-18840b4dc361 butto
      quickchartButtonEl.style.display =
        google.colab.kernel.accessAllowed ? 'block' : 'none';
    })();
  </script>
</div>

</div>
```



png

```
#  SHAP for SVM (kernel explainer) — may be slow
import shap
explainer = shap.KernelExplainer(svm_tuned.predict_proba, X_train_sm[:100])  #
shap_values = explainer.shap_values(X_test_scaled[:10])  # explain first 10
shap.summary_plot(shap_values, pd.DataFrame(X_test_scaled[:10], columns=featu
```

```
  0%|          | 0/10 [00:00<?, ?it/s]
```



png

```
import joblib

# Example: save tuned SVM (full)
joblib.dump(svm_tuned, "final_svm_model.pkl")
# and save scaler if not already saved
joblib.dump(scaler, "scaler.pkl")
print("Saved final_svm_model.pkl and scaler.pkl")
```

```
Saved final_svm_model.pkl and scaler.pkl
```